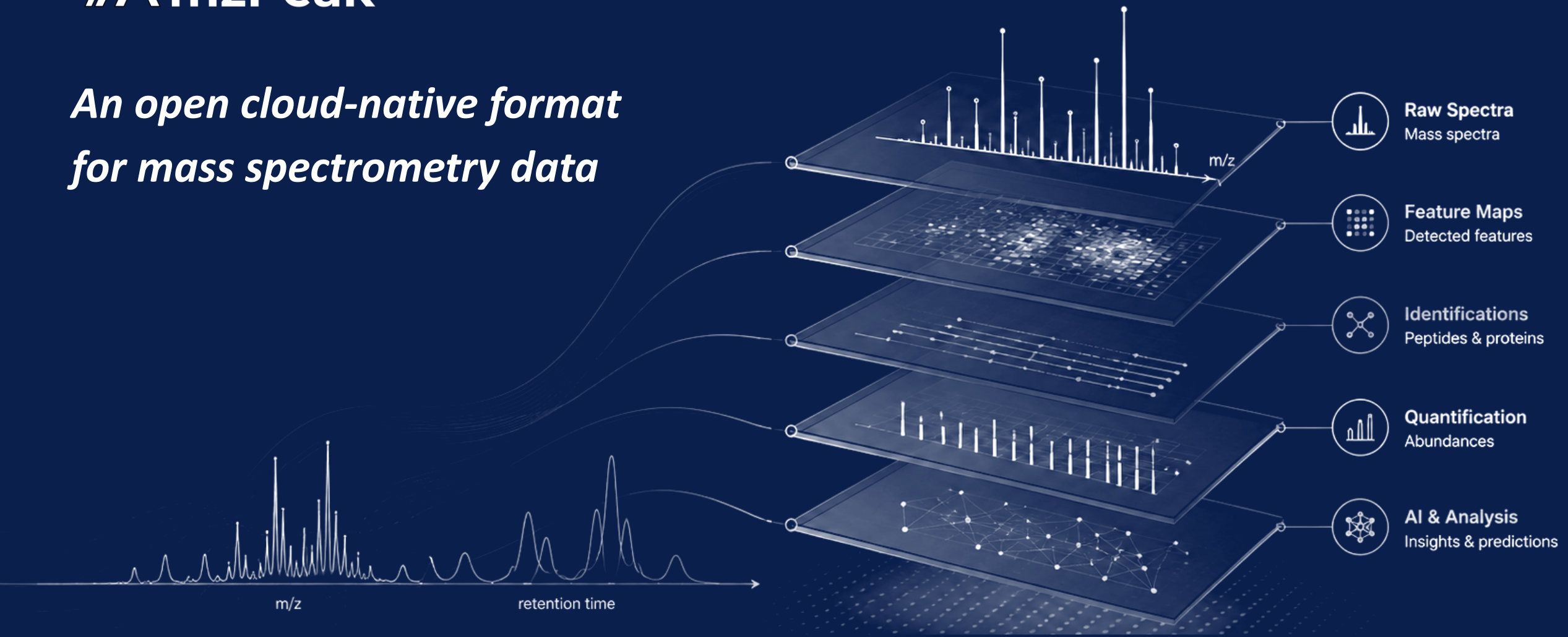
mzPeak

An open cloud-native format for mass spectrometry data



Contact: **info@mzpeak.org**

version 2026.11.03

The mzPeak core team

mzPeak



Tim Van Den Bossche



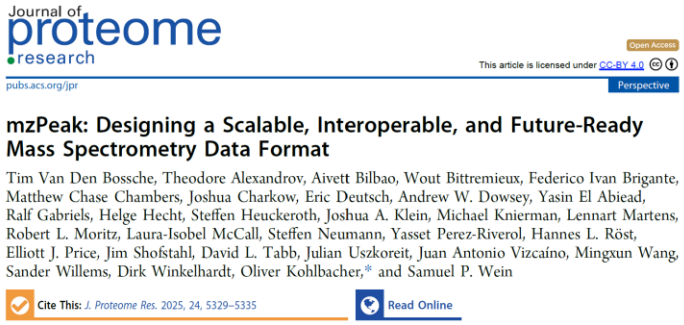
Joshua Klein



Oliver Kohlbacher



Sam Wein



Journal of proteome research

pubs.acs.org/jpr

Open Access

This article is licensed under CC-BY 4.0

Perspective

mzPeak: Designing a Scalable, Interoperable, and Future-Ready Mass Spectrometry Data Format

Tim Van Den Bossche, Theodore Alexandrov, Aivett Bilbao, Wout Bittremieux, Federico Ivan Brigante, Matthew Chase Chambers, Joshua Charkow, Eric Deutsch, Andrew W. Dowsey, Yasin El Abiead, Ralf Gabriels, Helge Hecht, Steffen Heuckeroth, Joshua A. Klein, Michael Knierman, Lennart Martens, Robert L. Moritz, Laura-Isobel McCall, Steffen Neumann, Yasset Perez-Riverol, Hannes L. Röst, Elliott J. Price, Jim Shofstahl, David L. Tabb, Julian Uszkoreit, Juan Antonio Vizcaíno, Mingxun Wang, Sander Willems, Dirk Winkelhardt, Oliver Kohlbacher,* and Samuel P. Wein

Cite This: *J. Proteome Res.* 2025, 24, 5329−5335

Read Online

Proteomics Standards Initiative

Technical committee members

White paper: https://pubs.acs.org/doi/10.1021/acs.jproteome.5c00435 **Draft specification:** https://www.mzpeak.org/spec

Existing formats were not built for today's data volumes

mzPeak

i Space-inefficient open formats

Verbose XML-based standards (i/mzML) result in massive file sizes that increase overall storage costs

ii Performance limitations

Legacy formats lack the high-performance random access needed for modern cloud-native workflows

iii Vendor-formats

Dependence on vendor-specific formats isolates data and hampers long-term accessibility

iv Multi-cloud compatibility

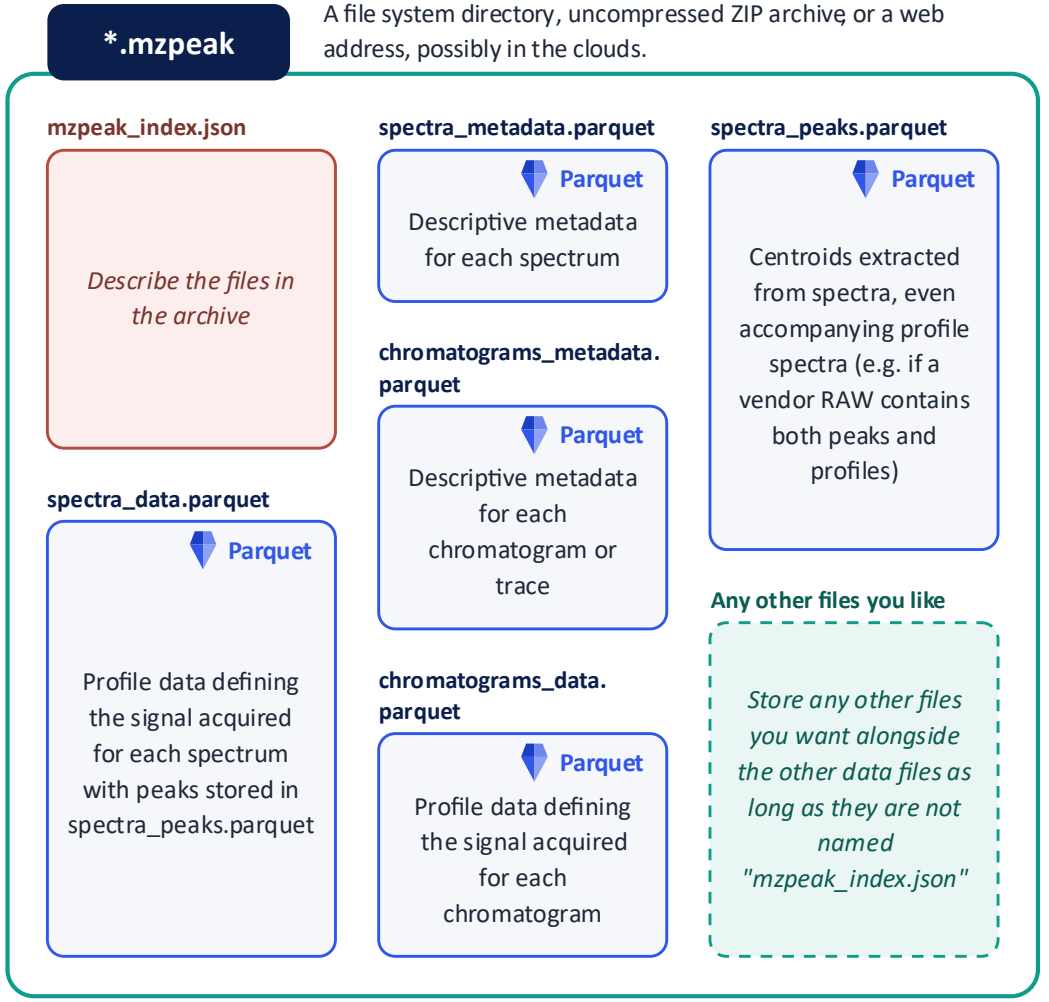
Outdated architectures prevent seamless scaling across distributed cloud environments

White paper: https://pubs.acs.org/doi/10.1021/acs.jproteome.5c00435 **Draft specification:** https://www.mzpeak.org/spec

mzPeak eliminates these barriers with a single open, cloud-native format

mzPeak

1. **Based on Apache Parquet**
   Compact, column-based storage; 2-4× smaller than mzML on average, on average smaller than vendor data
2. **Native random access**
   Load only the data you need, from local storage or cloud
3. **Open standard under HUPO-PSI**
   Public specification; all instrument types; extensible
4. **Designed for cloud object storage**
   S3-compatible; no conversion or custom parser needed



White paper: https://pubs.acs.org/doi/10.1021/acs.jproteome.5c00435 **Draft specification:** https://www.mzpeak.org/spec

mzPeak resolves the core limitations of both mzML and vendor formats

mzPeak

Limitations of mzML	Solutions of mzPeak
Large file sizes and slow data access	Compact binary storage with random access
Limited support for multidimensional data (ion mobility, MS imaging)	Native support for multi-dimensional workflows
Limited metadata capture and extensibility	Comprehensive, extensible metadata integration (SDRF-Proteomics, encryption support)
Limitations of vendor formats	
Proprietary, closed ecosystems	Open format with public specification and vendor-independent design
Lack of long-term archival guarantees	Versioned, open, and community-maintained for archival durability
Limited cross-platform and software interoperability	Broad accessibility through open reference implementations and multi-language bindings

White paper: https://pubs.acs.org/doi/10.1021/acs.jproteome.5c00435 **Draft specification:** https://www.mzpeak.org/spec

Benefits of ONE common format

mzPeak



Reduced Development Overhead

Streamline integration through standardized I/O interfaces, minimizing custom engineering efforts.



Cloud-Readiness

Accelerate deployment of modern service models and remote management capabilities.



Influence Standards

Shape the evolution of the protocol to ensure it meets high-performance, industrial-scale throughput requirements.



Competitive Differentiation

Position products within a broader ecosystem to increase market reach and customer retention.

White paper: https://pubs.acs.org/doi/10.1021/acs.jproteome.5c00435 **Draft specification:** https://www.mzpeak.org/spec

mzPeak - smaller, faster, ready for integration

mzPeak

~ 2-4 x

smaller than mzML on average,
on average smaller than vendor data

< 2 s

to open 3 GB file from cloud

7

reference implementations

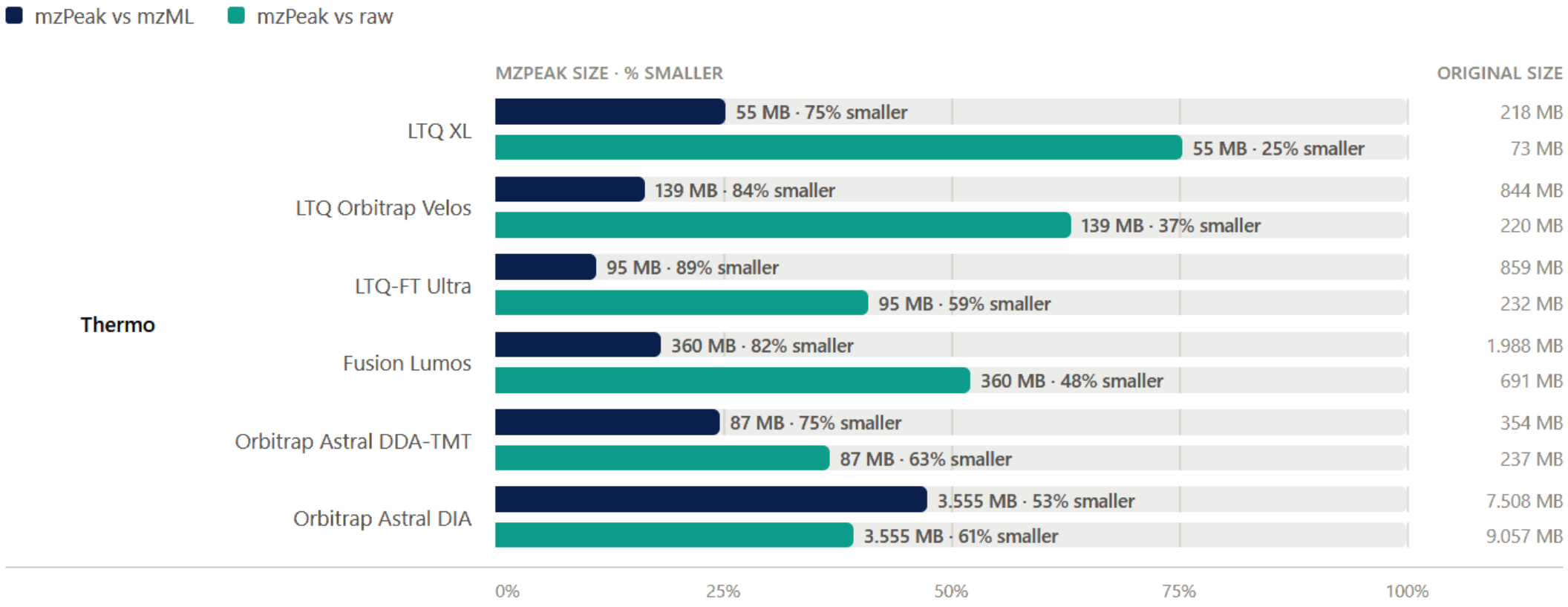
(Rust, Python, R, C#, TypeScript, Java, C++)

1

file per run

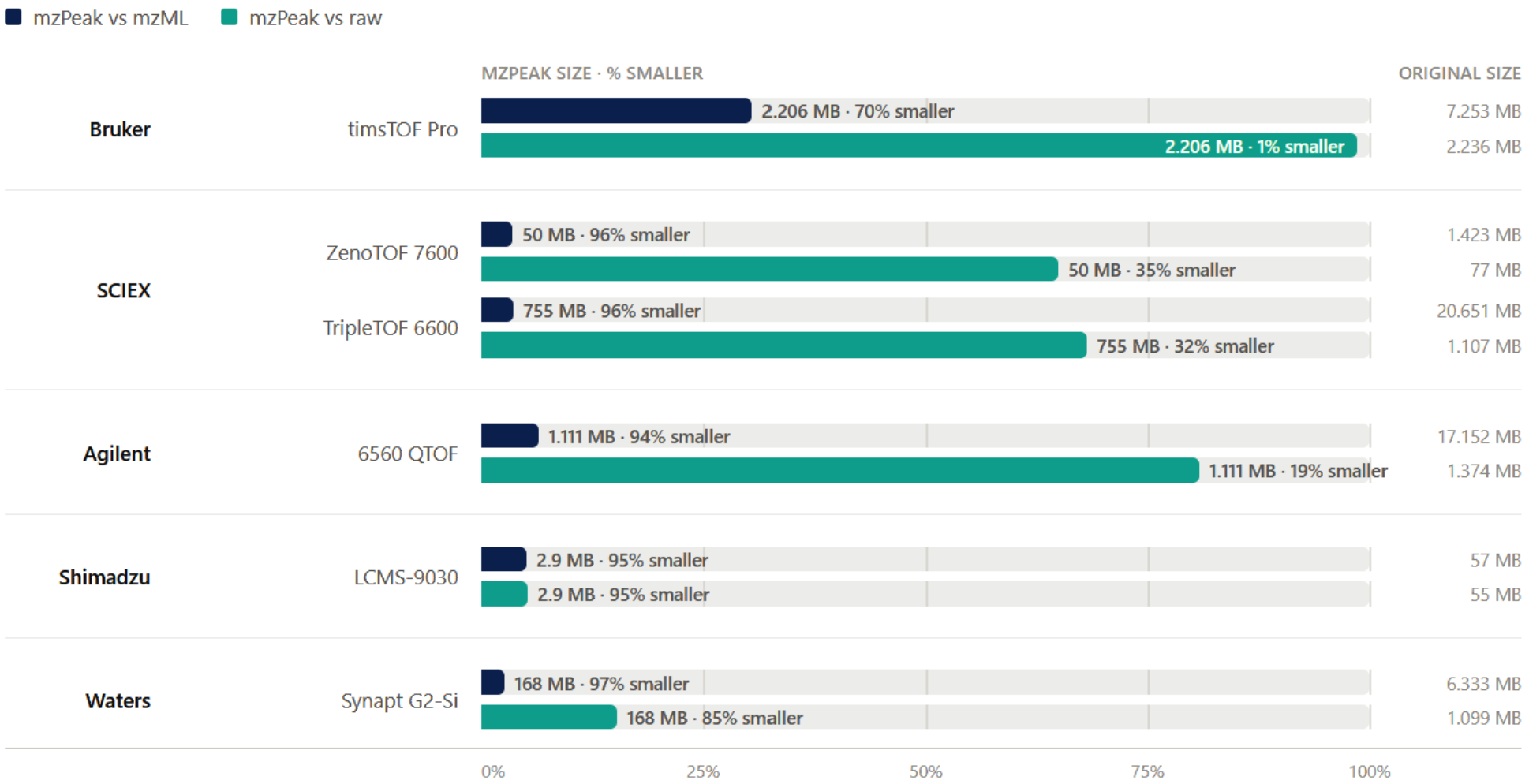
White paper: https://pubs.acs.org/doi/10.1021/acs.jproteome.5c00435 **Draft specification:** https://www.mzpeak.org/spec

mzPeak saves space



White paper: https://pubs.acs.org/doi/10.1021/acs.jproteome.5c00435 **Draft specification:** https://www.mzpeak.org/spec

mzPeak saves space

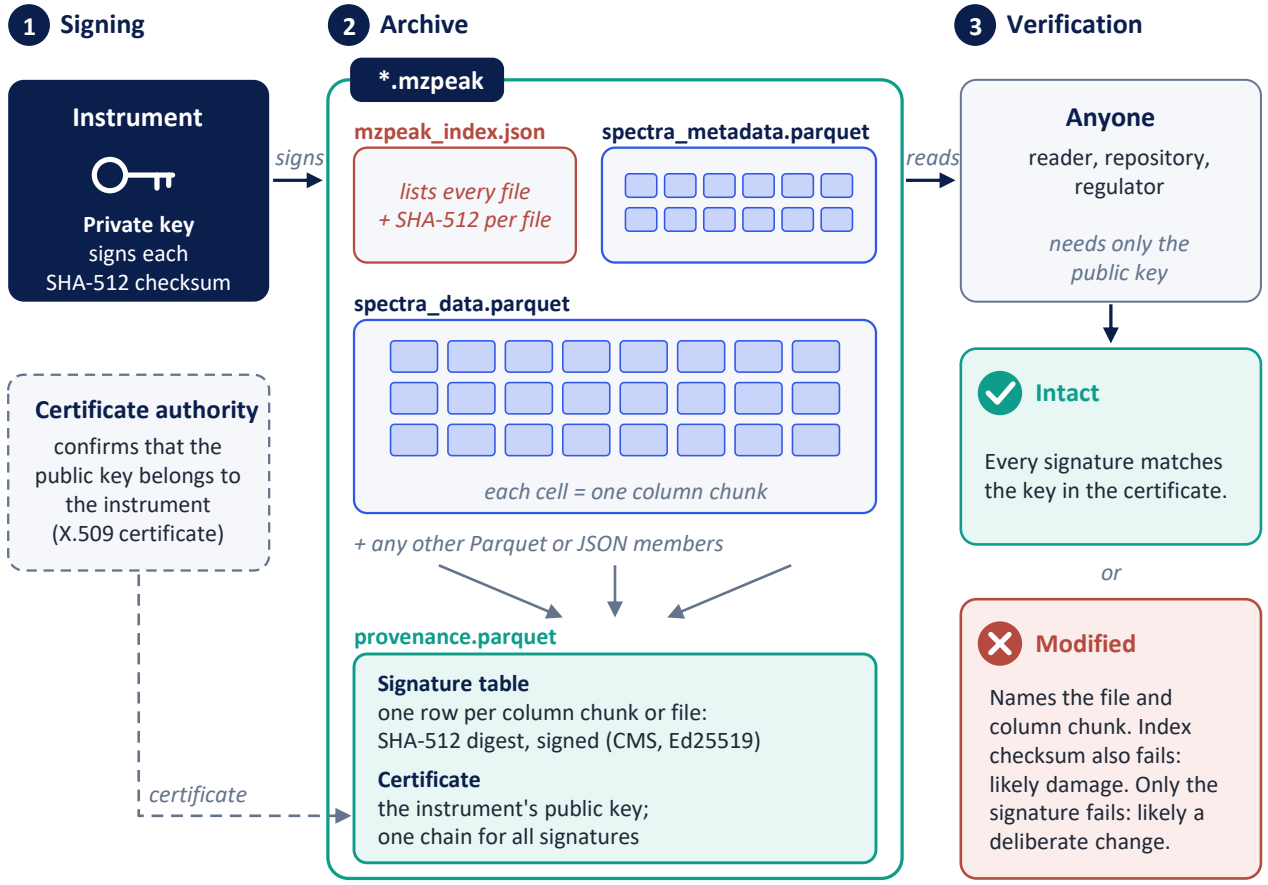


White paper: https://pubs.acs.org/doi/10.1021/acs.jproteome.5c00435 **Draft specification:** https://www.mzpeak.org/spec

Signed provenance: verify that a file is unchanged since it was signed

mzPeak

1. **Checksums alone are not enough**
   The index holds a SHA-512 per file, but whoever edits the data can also edit the index
2. **The instrument signs the checksums**
   Each SHA-512 digest of a file or column chunk is signed separately with the instrument's private key
3. **Open, standard cryptography**
   Ed25519 signatures in CMS (RFC 5652), with an X.509 certificate; one certificate chain per instrument
4. **Anyone can verify, and see what failed**
   Verification needs only the public key; a failed check names the file and column chunk



White paper: https://pubs.acs.org/doi/10.1021/acs.jproteome.5c00435 **Draft specification:** https://www.mzpeak.org/spec

Vendor metadata embedding

A Thermo example: 20240224_L3.raw - an Orbitrap Astral file from PXD061698

5.57 GB

.raw (original vendor file)

3.24 GB

.mzPeak with all metadata (58% of .raw)

All diagnostic traces and logs preserved: background signal distribution arrays, peak metadata flags, instrument-calculated resolution, charge state estimates, and other channels stored in the .raw file.

Parquet files can be signed and/or encrypted to protect sensitive data or ensure authenticity.

Structure

Parquet members and column footers

Metadata | Structure

Archive members

MANIFEST mzpeak_index.json View JSON →	547 B
chromatograms_data.parquet	4.6 MB
chromatograms_metadata.parquet	73.8 KB
spectra_data.parquet	469.4 MB
spectra_metadata.parquet	9.3 MB
spectra_peaks.parquet	2.77 GB
spectrum_noise_data.parquet	1.3 MB

spectra_peaks.parquet

328,554,226 rows · **8** columns · **314** row groups · 2.77 GB compressed / 4.59 GB raw (1.7×) · parquet-cpp-arrow version 23.0.0

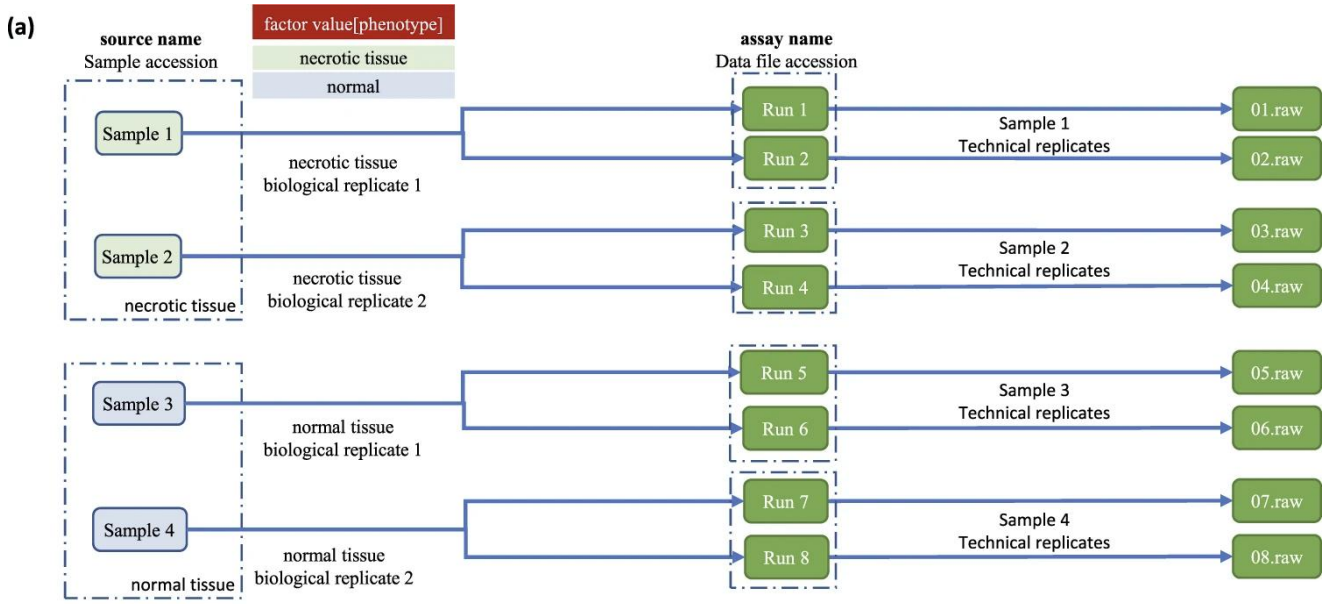
row groups: **314** · sizes min 5.7 MB / med 15.0 MB / max 15.9 MB · page index: **yes** · 1303.8 chunks/spectrum

column	type	codec	values	size	share
▸ point.spectrum_index	INTEGER	ZSTD	328,554,226	2.2 MB	0%
▸ point.mz	DOUBLE	ZSTD	328,554,226	834.2 MB	29%
▸ point.intensity	FLOAT	ZSTD	328,554,226	1.26 GB	46%
▸ point.baseline	FLOAT	ZSTD	328,554,226	33.7 MB	1%
▸ point.noise	FLOAT	ZSTD	328,554,226	33.0 MB	1%
▸ point.peak_option_flags	INT32	ZSTD	328,554,226	1.1 MB	0%
▸ point.resolution	INT32	ZSTD	328,554,226	582.3 MB	21%
▸ point.charge	INT32	ZSTD	328,554,226	58.7 MB	2%

https://github.com/HUPO-PSI/mzPeak.NET.git -- dotnet run --project "MZPeakNet.AppTest" -- thermo $infile.raw $outfile.mzpeak -- works out of the box with dotnet 8.0

mzPeak integrates with study metadata

- Integration with **SDRF-Proteomics** (HUPO-PSI standard; Dai *et al.* 2021. Nature Communications, https://doi.org/10.1038/s41467-021-26111-3) can track sample information even for complex multi-omics experiments
- SDRF stores the **study metadata**, but mzPeak can embed **run metadata**, and keeps it consistent with the study design
- mzPeak files embed the parts of the sample information relevant to the run
- Information for channel assignment in TMT, for example, is thus tracked where it belongs and allows self-contained analyses

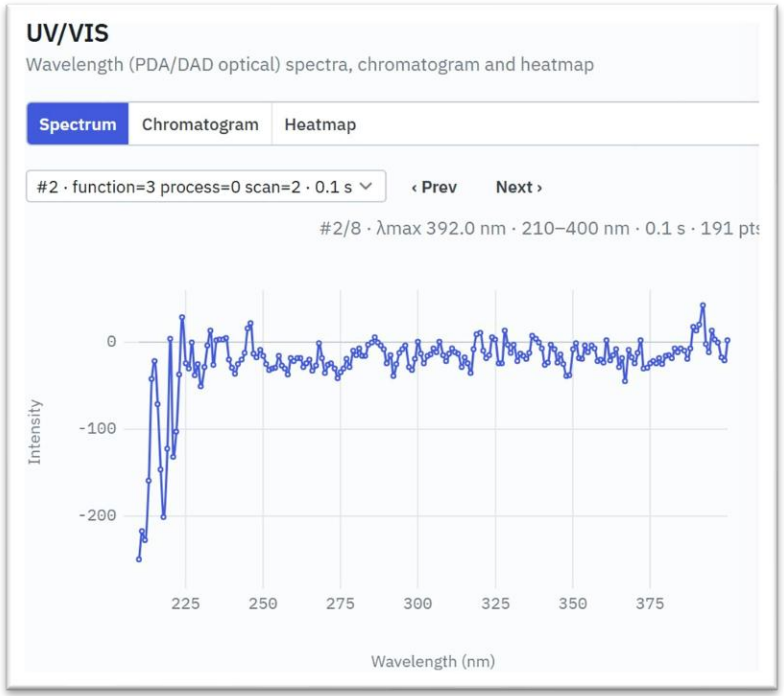


(b)

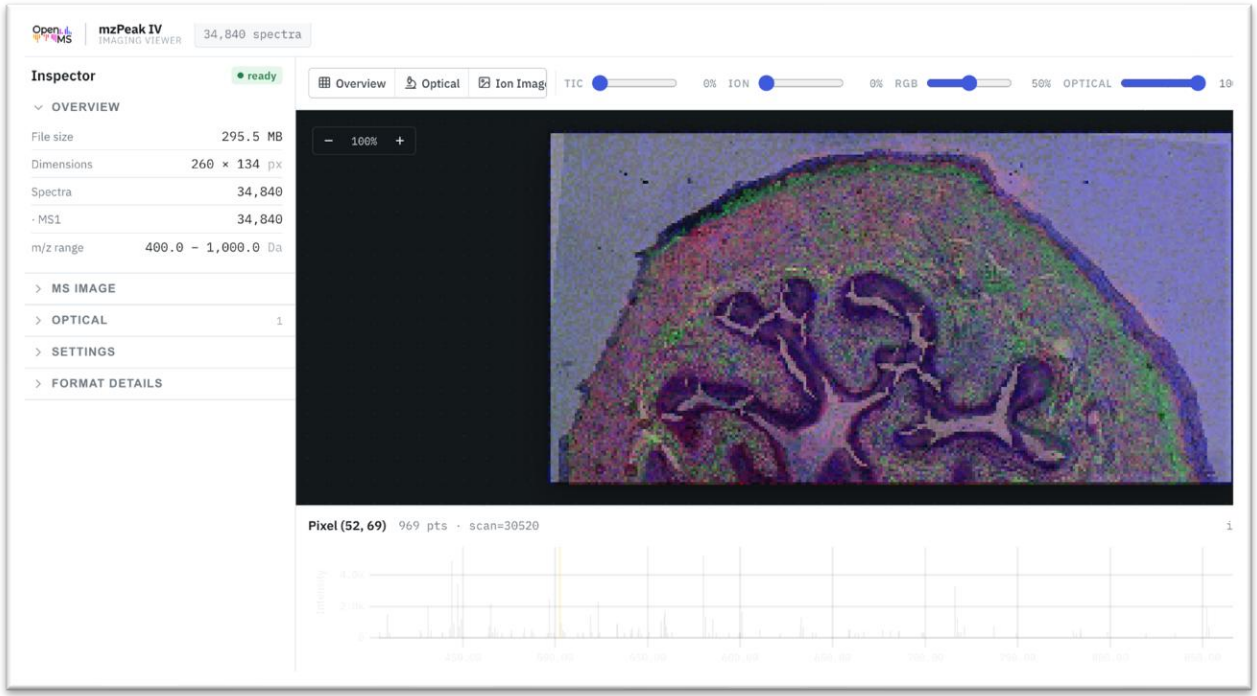
	Sample properties						Data file properties					Study variables
source name	characteristics [organism]	…	characteristics [disease]	characteristics [phenotype]	characteristics [biological replicate]	assay name	comment [fraction identifier]	comment [label]	comment [technical replicate]	…	comment [data file]	factor value [phenotype]
Sample 1	homo sapiens	…	liver cancer	necrotic tissue	1	Run 1	1	label free sample	1	…	01.raw	necrotic tissue
Sample 1	homo sapiens	…	liver cancer	necrotic tissue	1	Run 2	1	label free sample	2	…	02.raw	necrotic tissue
Sample 2	homo sapiens	…	liver cancer	necrotic tissue	2	Run 3	1	label free sample	1	…	03.raw	necrotic tissue
Sample 2	homo sapiens	…	liver cancer	necrotic tissue	2	Run 4	1	label free sample	2	…	04.raw	necrotic tissue
Sample 3	homo sapiens	…	liver cancer	normal	1	Run 5	1	label free sample	1	…	05.raw	normal
Sample 3	homo sapiens	…	liver cancer	normal	1	Run 6	1	label free sample	2	…	06.raw	normal
Sample 4	homo sapiens	…	liver cancer	normal	2	Run 7	1	label free sample	1	…	07.Raw	normal
Sample 4	homo sapiens	…	liver cancer	normal	2	Run 8	1	label free sample	2	…	08.raw	normal

Other data modalities can be embedded (imaging, UV/VIS, …)

- No sidecar files — MS data, metadata, optical images and UV/VIS traces are all embedded
- More compact than imzML (20-50% of the size)
- Multiple spectra per pixel and multiple optical images per run
- Efficient interactive access to the data



UV/VIS spectrum view



MSI ion image with embedded optical image

mzPeakViewer – demo

mzPeak

mzPeak Governance Tools Try it Supporters Contact Specification

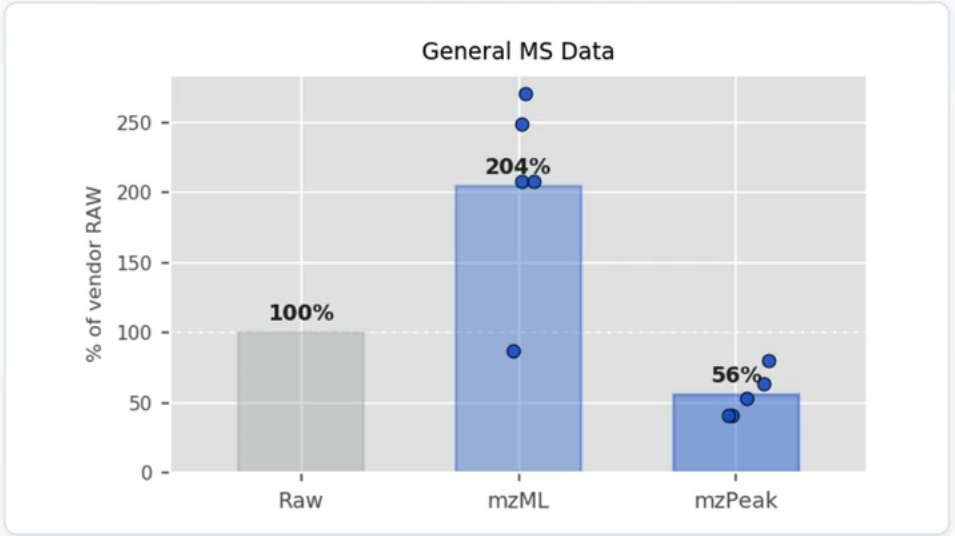
OPEN STANDARD (UNDER DEVELOPMENT) · HUPO-PSI

The modern mass spectrometry data format

Compact, fast, and cloud-native — a Parquet-in-ZIP successor to mzML. Losslessly smaller, and randomly addressable straight over HTTP.

Read the white paper | Browse live examples

Governed by HUPO-PSI | Reference implementations across seven languages



Smaller than mzML, losslessly

mzPeak is cloud-native and AI-ready

mzPeak

Cloud-native format

- mzPeak is designed for cloud object storage
- Direct integration with data lake infrastructures
- Seamless integration into large-scale, multi-tiered storage systems saves money
- Random access to the data means that only the data that is needed is requested
- Saves egress fees for cloud storage

AI readiness

- Built on Apache Parquet, the native format of modern AI/ML data stacks
- Natively supported by Pandas, Polars, Spark, Arrow & data loaders - no binary parser needed
- Columnar layout enables efficient feature extraction - read only the columns needed
- Random access streams for training directly from the data lake
- Rich structured metadata gives models the context for self-describing training data

Spec v0.9 Is ready - HUPO-PSI standardization starts fall of 2026

mzPeak

- Community white paper published (Van Den Bossche T et al., Journal of Proteome Research, 2025, DOI: 10.1021/acs.jproteome.5c00435)
- Reference implementations available in seven programming languages (Rust, R, C#, Python, TypeScript, Java, C++)
- Example datasets already converted for Thermo, Bruker, SCIEX, Waters, Agilent, and Shimadzu instruments — data.mzpeak.org
- Public specification available at mzpeak.org/spec
- Seeking feedback from instrument vendors and software developers

White paper: https://pubs.acs.org/doi/10.1021/acs.jproteome.5c00435 **Draft specification:** https://www.mzpeak.org/spec

mzPeak roadmap (2026 – 2027)



Nov 2026

v1.0 freeze specification

- Finalize specification
- Start PSI document process
- **Vendor feedback deadline**

Apr 2027

HUPO-PSI approval

- Approval by HUPO-PSI of the specification document
- Implementations can be finalised against the approved spec doc

Jun 2027

Presentation

- Present at ASMS
- **Public announcement of the standard**

White paper: https://pubs.acs.org/doi/10.1021/acs.jproteome.5c00435 **Draft specification:** https://www.mzpeak.org/spec

What mzPeak offers to vendors

1. **Open, free specification** no licensing fees or proprietary dependencies
2. **Direct influence over the standard** through HUPO-PSI participation
3. **Cloud and AI readiness out of the box,** without additional engineering
4. **Interoperability & seamless integration** with the broader MS ecosystem from day one

White paper: doi.org/10.1021/acs.jproteome.5c00435 **Draft specification:** mzpeak.org/spec

What vendors should contribute now

1. **Provide instrument-specific feedback on the draft spec as soon as possible**
   Particularly around vendor metadata, compression behaviour, and edge cases for your data types
   → please send us over some of your edge cases

2. **Designate a technical contact**
   For spec review, implementation support, and ongoing collaboration

3. **Commit to implementing mzPeak read/write support**
   Targeting availability by ASMS 2027, aligned with the public launch of the standard

White paper: doi.org/10.1021/acs.jproteome.5c00435 **Draft specification:** mzpeak.org/spec

Thank you & acknowledgements

mzPeak

- **mzPeak is a community effort** — developed openly with contributors from academia, instrument vendors, and software projects
- **Specification and white paper**: the mzPeak working group and all co-authors (Van Den Bossche T et al., Journal of Proteome Research, 2025)
- **Standardization**: the HUPO-PSI Mass Spectrometry Standards working group
- **Reference implementations**: various contributors, incl. Oliver Kohlbacher, Joshua Klein & Peter Jones
- **Software adopters**: CompOmics, OpenMS, Skyline, MSFragger, … *(dedicated EuBIC hackathon in Jan. 27)*
- **Vendor adopters:** Bruker and Shimadzu

BRUKER

SHIMADZU
Excellence in Science

Get involved:
https://mzpeak.org · info@mzpeak.org

mzPeak

Additional Resources

mzPeak Website: https://mzpeak.org

mzPeak Whitepaper: https://pubs.acs.org/doi/10.1021/acs.jproteome.5c00435

mzPeak draft specification: https://mzpeak.org/spec

Example data: https://data.mzpeak.org/v09/index.html

Interact with the mzPeak community

Contact us: **info@mzpeak.org**

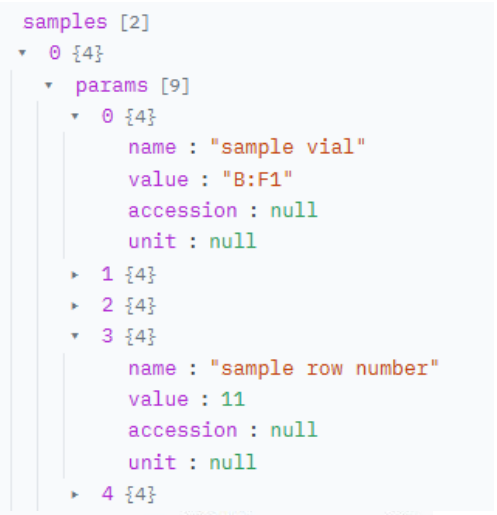
Vendor Metadata Embedding

mzPeak

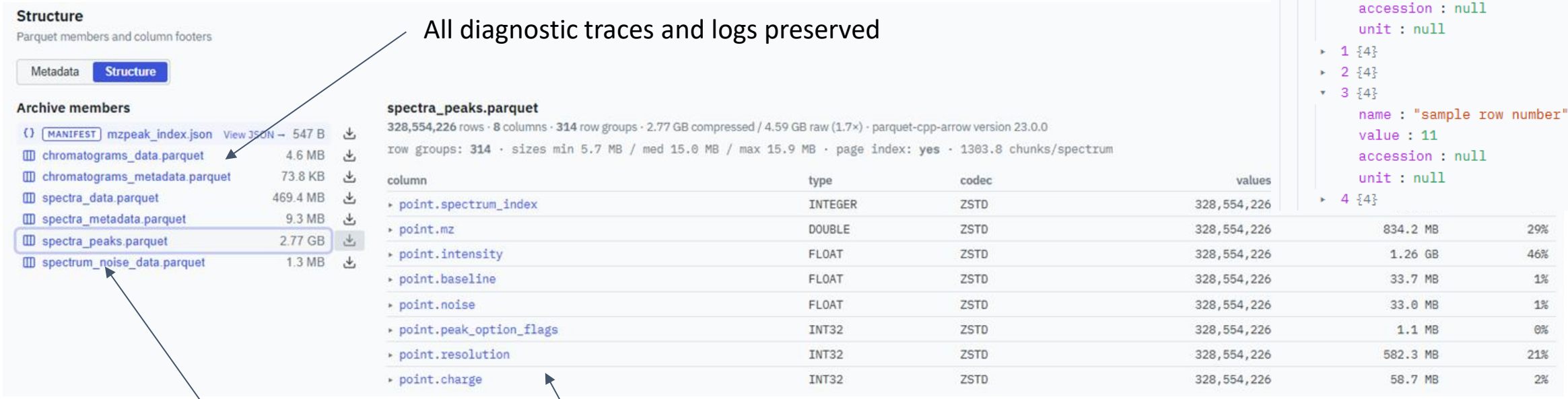
A Thermo example: 20240224_L3.raw - an Orbitrap Astral file from PXD061698

- .raw - 5.57GB
- .mzPeak with all metadata - 3.24GB (58% of .raw)

Sample metadata from acquisition software



All diagnostic traces and logs preserved



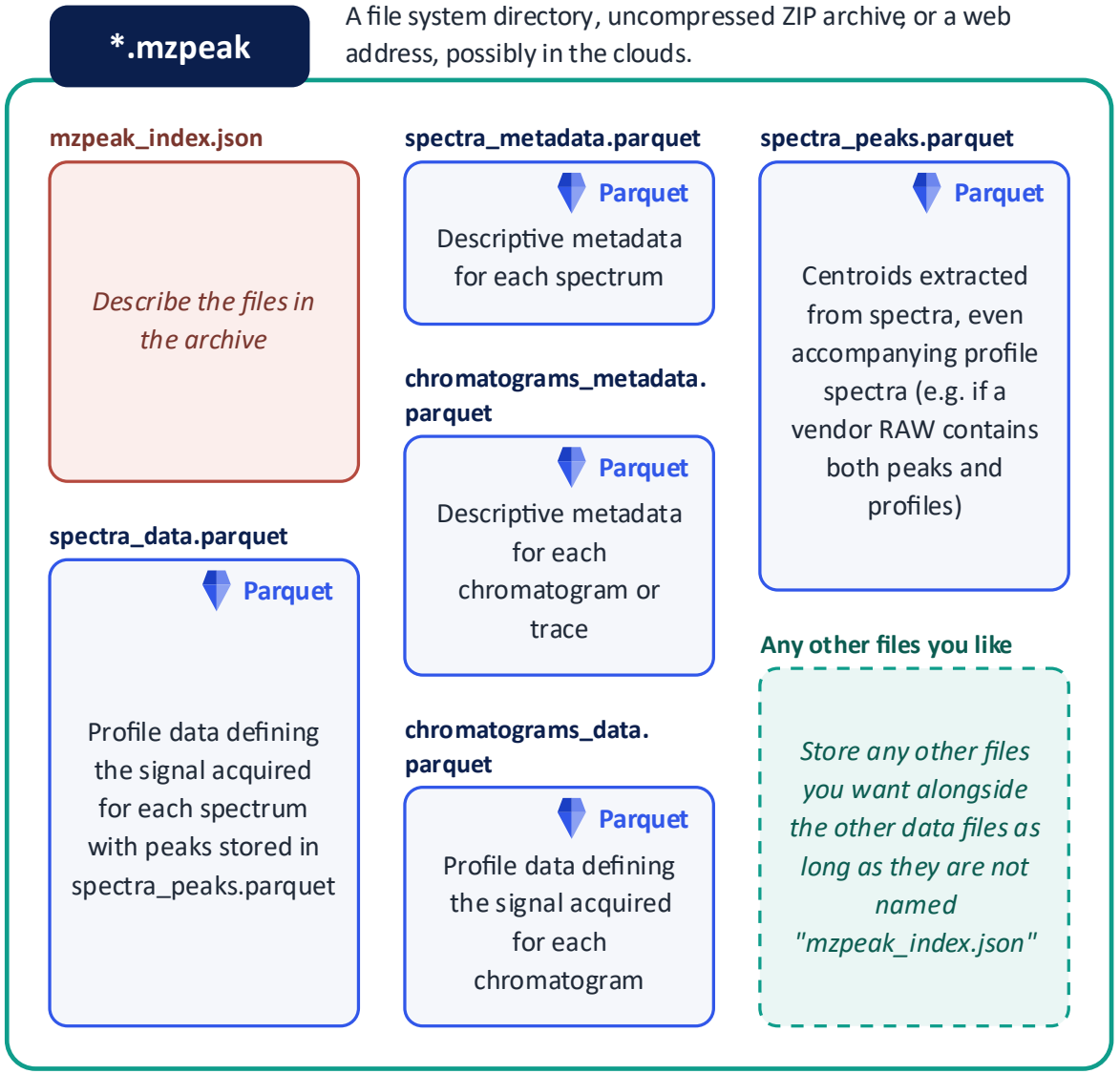
Background signal distribution arrays for every spectrum

Peak metadata flags, instrument-calculated resolution, charge state estimates, and other channels stored in the .raw file

https://github.com/HUPO-PSI/mzPeak.NET.git `dotnet run --project "MZPeakNet.AppTest" -- thermo $infile.raw $outfile.mzpeak` works out of the box with dotnet 8.0

Anatomy of an mzPeak File

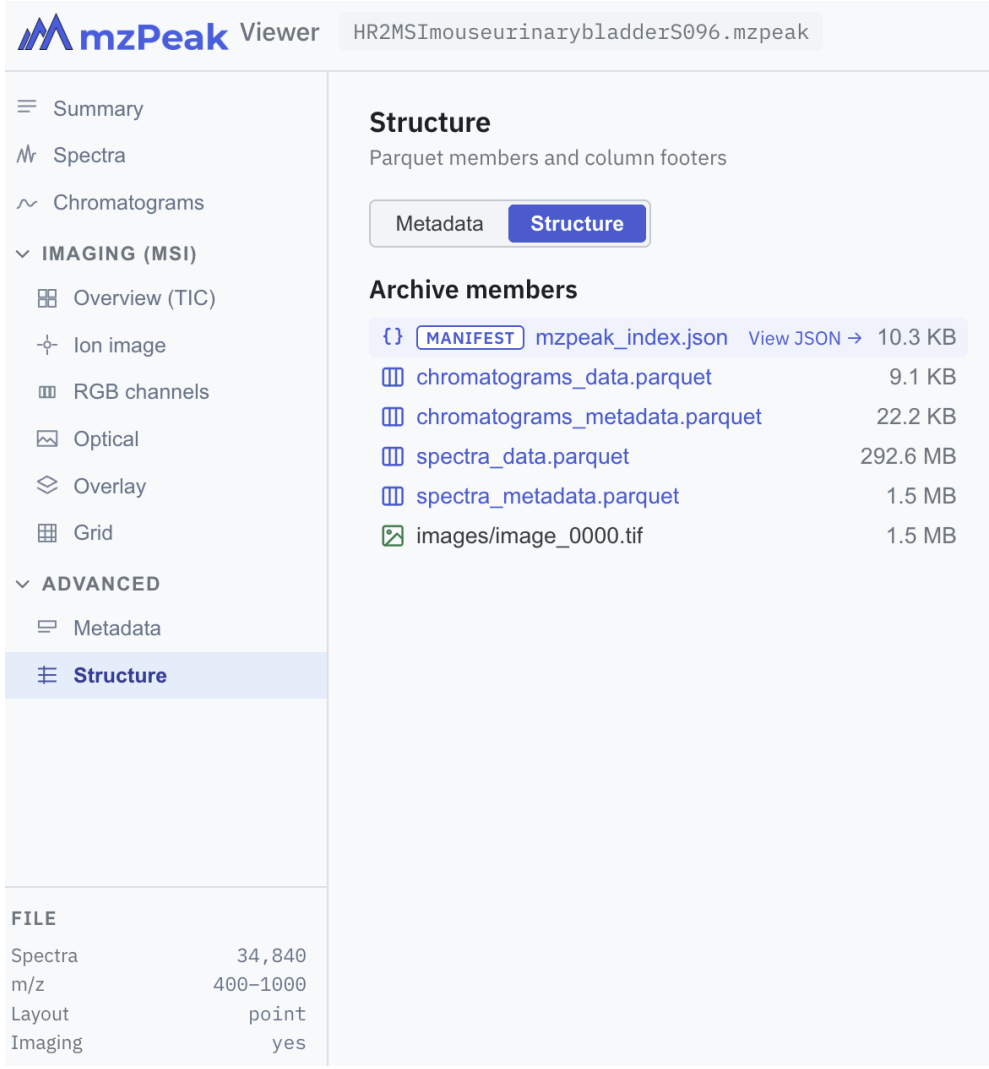
- ONE file for all data related to a run
- Based on Apache Parquet
- Space-efficient column-based format for spectra and chromatograms
- Cloud-enabled (S3-compatible)
- Standardized metadata (HUPO-PSI)
  - JSON for global metadata
  - Efficient spectrum-wise metadata in Parquet
- Native support for encryption of any parts of the file
- ZIP container can contain arbitrary additional data (e.g., optical images)
- Lossless compression (optional lossy compression)



mzPeak Is Compact and Self-Contained

mzPeak contains everything belonging to a run

- Each mzPeak file is a container with different types of payload
- No sidecar files, no directories with additional files
- Optical images for MSI can be embedded
- Study design can be embedded
- Vendor-specific information (methods) can be embedded
- Users can inject/remove their own content with existing tools

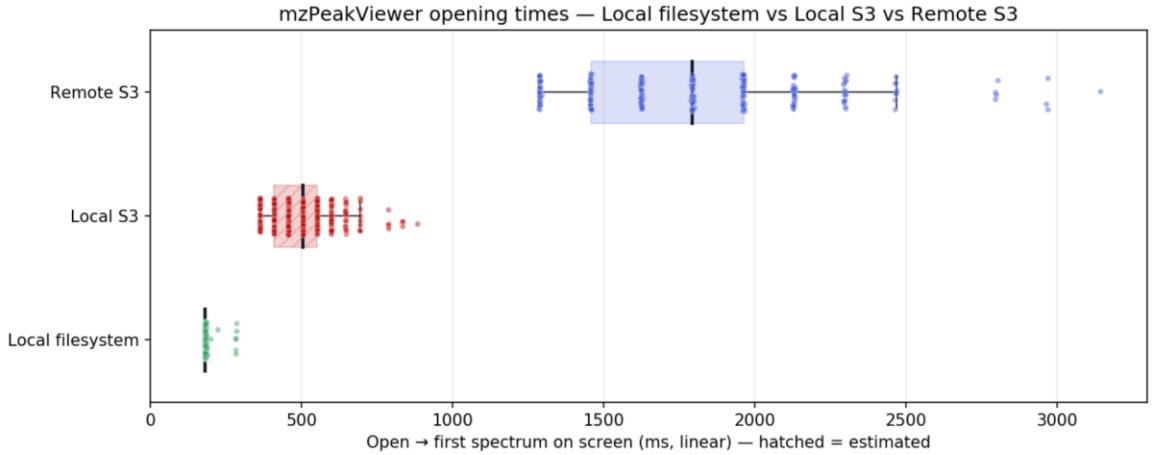


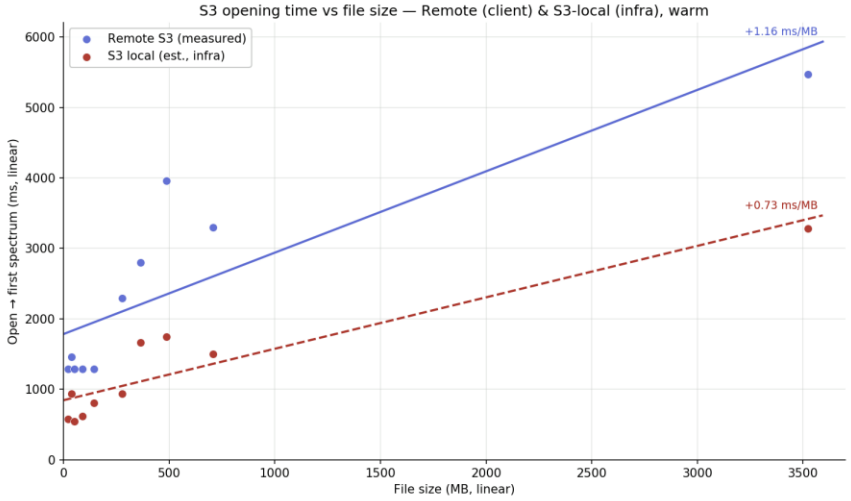
mzPeak Is Fast

Random access and partial access

- Task: opening large file (3 GB) and showing overview information, displaying an arbitrary spectrum
- Local access (avg. 0.2 s) is instantaneous
- Local S3 (same network) is very fast (0.5 s)
- Remote S3 access is still fast (avg. 1.8 s), but network latency slows down access
- Access time for overview and individual spectra grows very slightly with file size

Note: *Data based on browser-based access to local storage on a laptop vs cloud access (S3 hosted at StackIT); S3 times vary mostly based on network connection quality; times shown are times from entering URL to display of the first MS spectrum.*





mzPeak Simplifies Data Sharing

mzPeak

Clickable links in papers or archive reference mzPeak file including a deep link to a spectrum

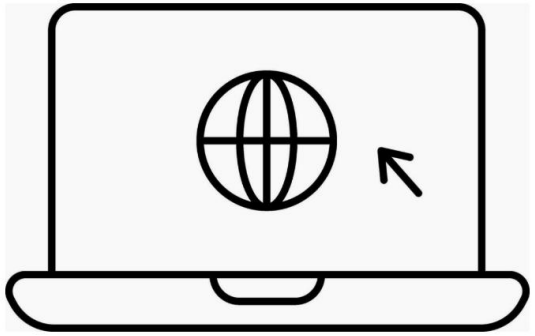
URL references spectrum XIC or similar, e.g., mzpeak.org/view/<mzpeakfile>?scan=<scan no.>

Paper

Archive

WWW

Opening the URL starts viewer app from mzpeak.org and data from the cloud-hosted mzPeak file

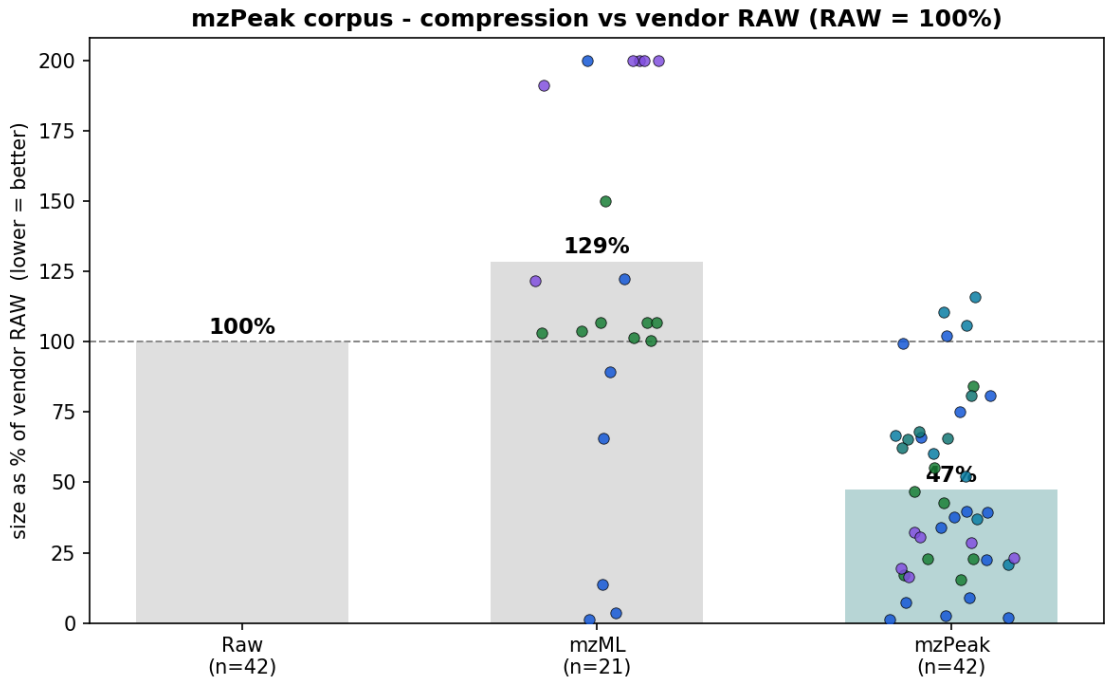


mzPeak

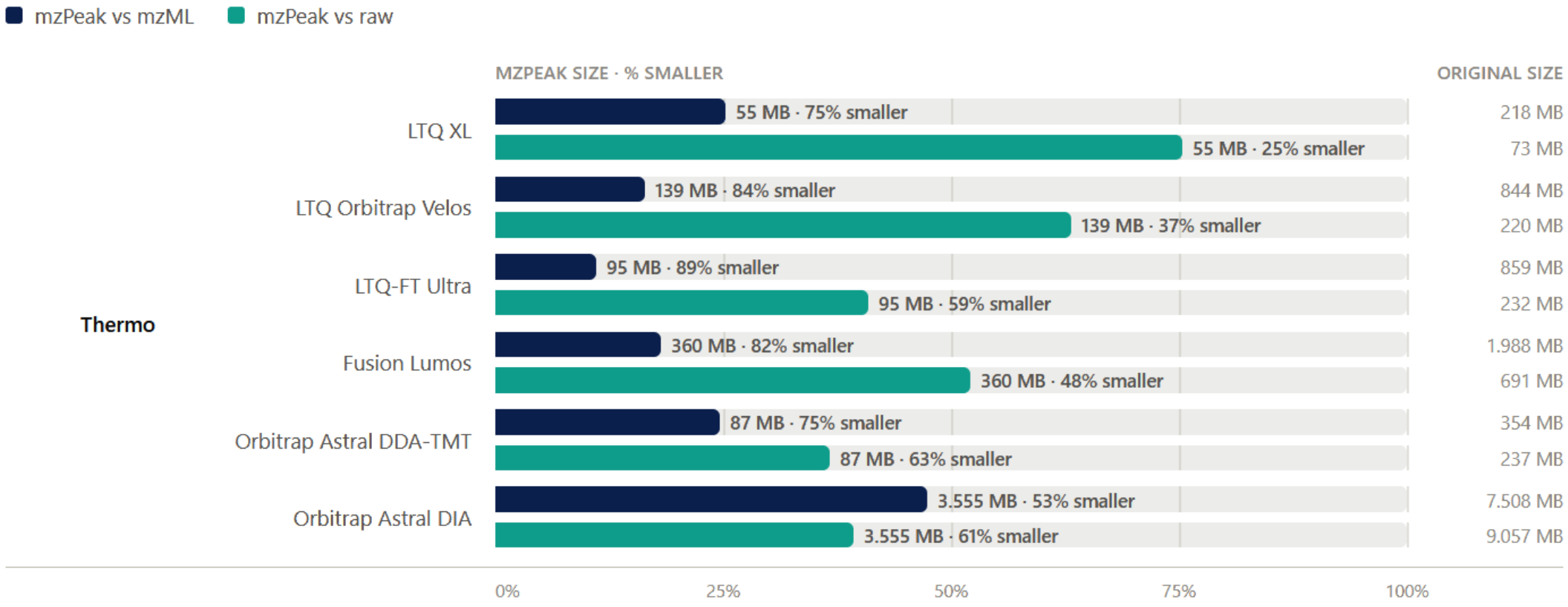
Everything stays inside the browser and data is displayed within seconds

mzPeak Saves Space

- mzML inflates file size relative to raw formats; mzPeak reduces it
- Space-saving features
  - Column-based format
  - Compact binary representation
  - Integrated supported for lossy and lossless compression (numpress)
- **Preliminary tests: mzPeak is typically 2-4x smaller than mzML; on average smaller than vendor data**
- For already-compressed vendor data (e.g. timsTOF), sizes are equivalent, as visible in the following charts

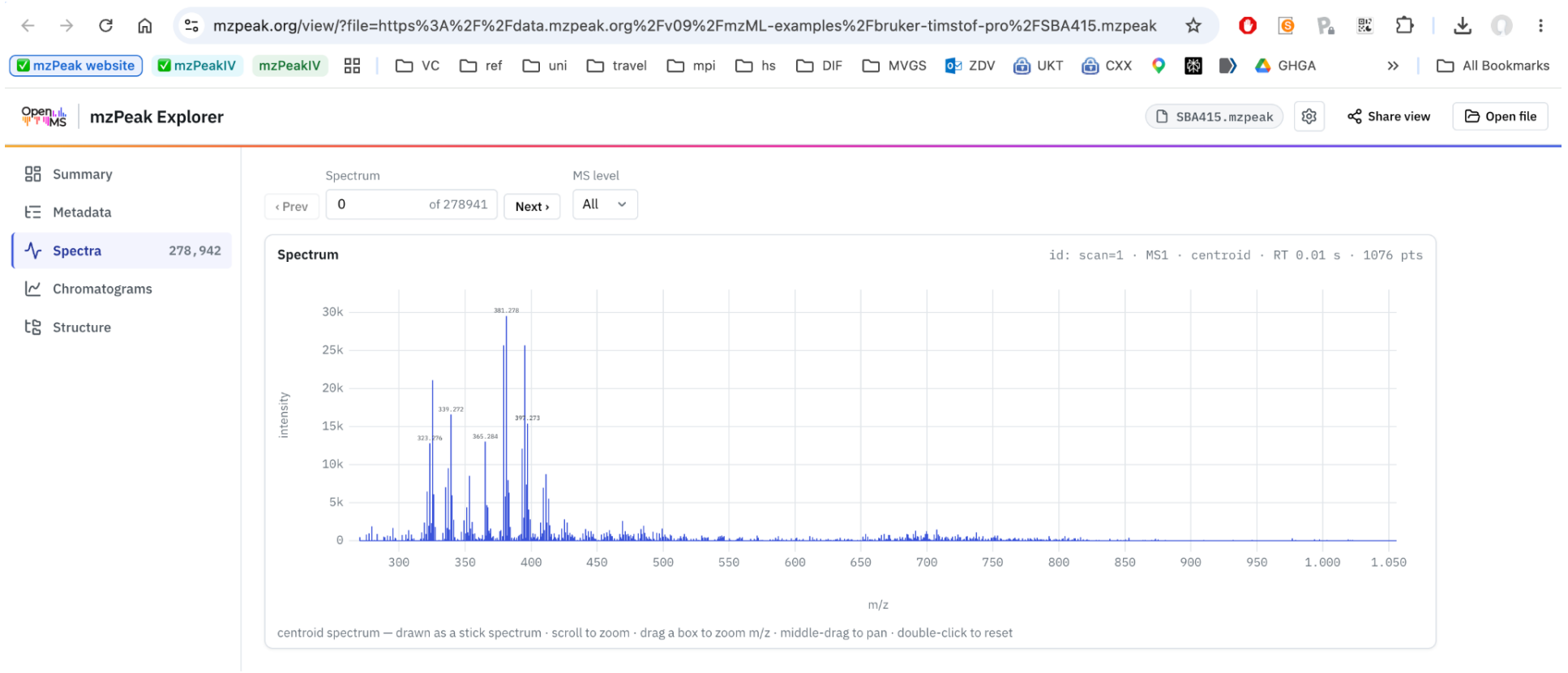


mzPeak Saves Space

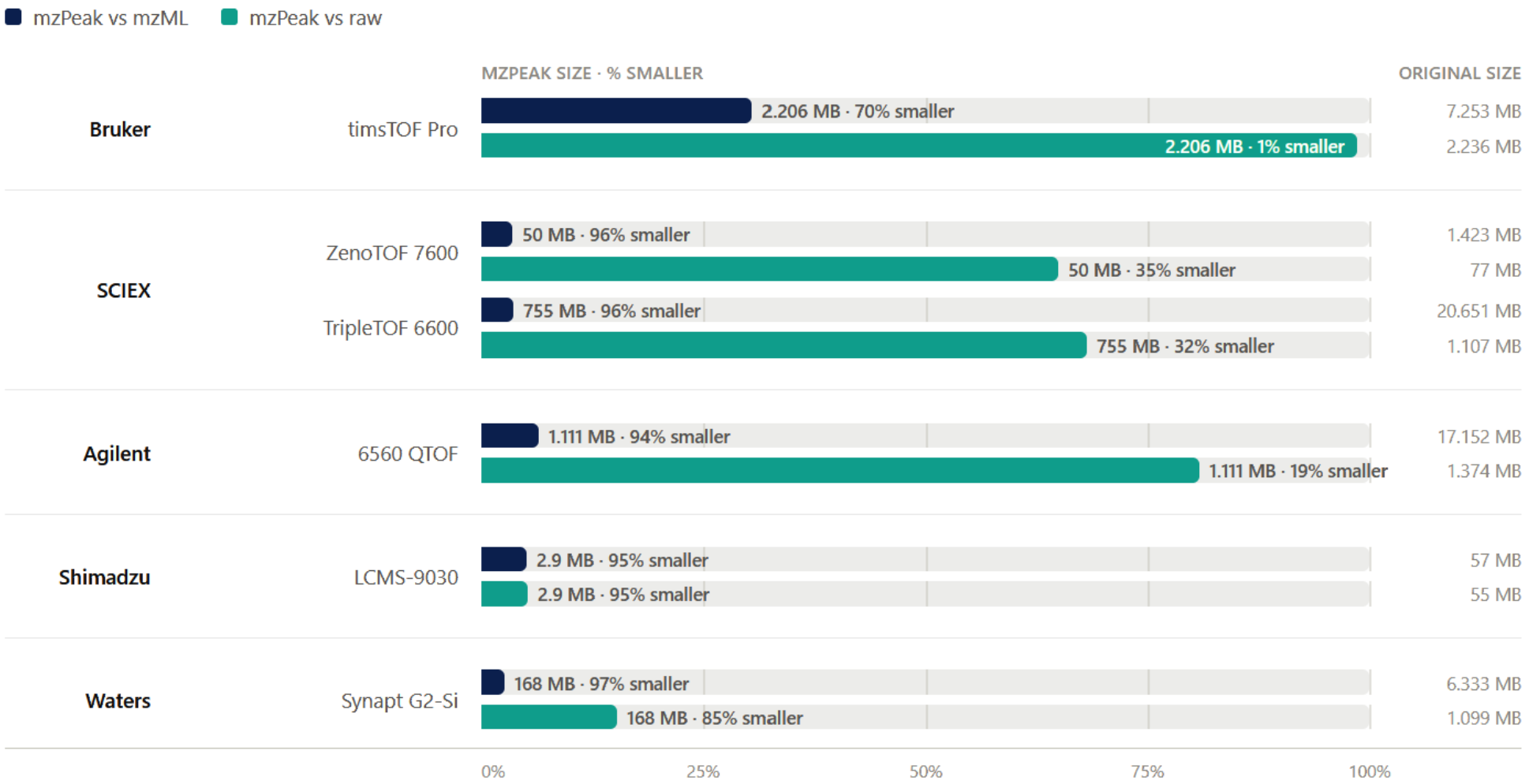


mzPeak Simplifies Data Sharing

- Random access and cloud nativity simplify link-based sharing of data
- Archives or papers can simply reference spectra, XICs, ion images etc. from the raw data directly
- mzPeakViewer can act as a resolver for cloud-stored data: mzpeak.org/view/<mzPeak URL>
- Displays the file natively in the browser, even on tablets or phones



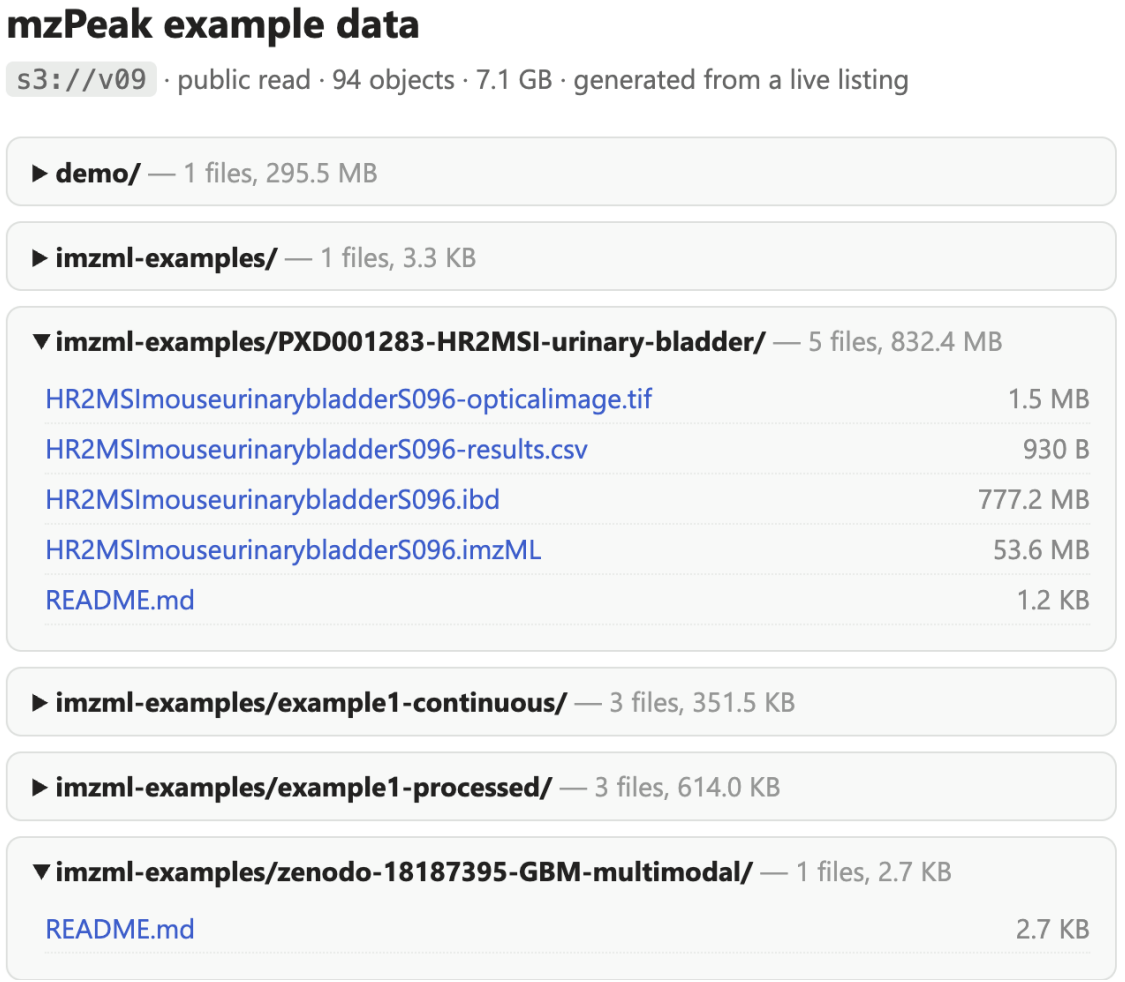
mzPeak Saves Space



Cloud-Hosted Content Can Be Accessed via Web

mzPeak

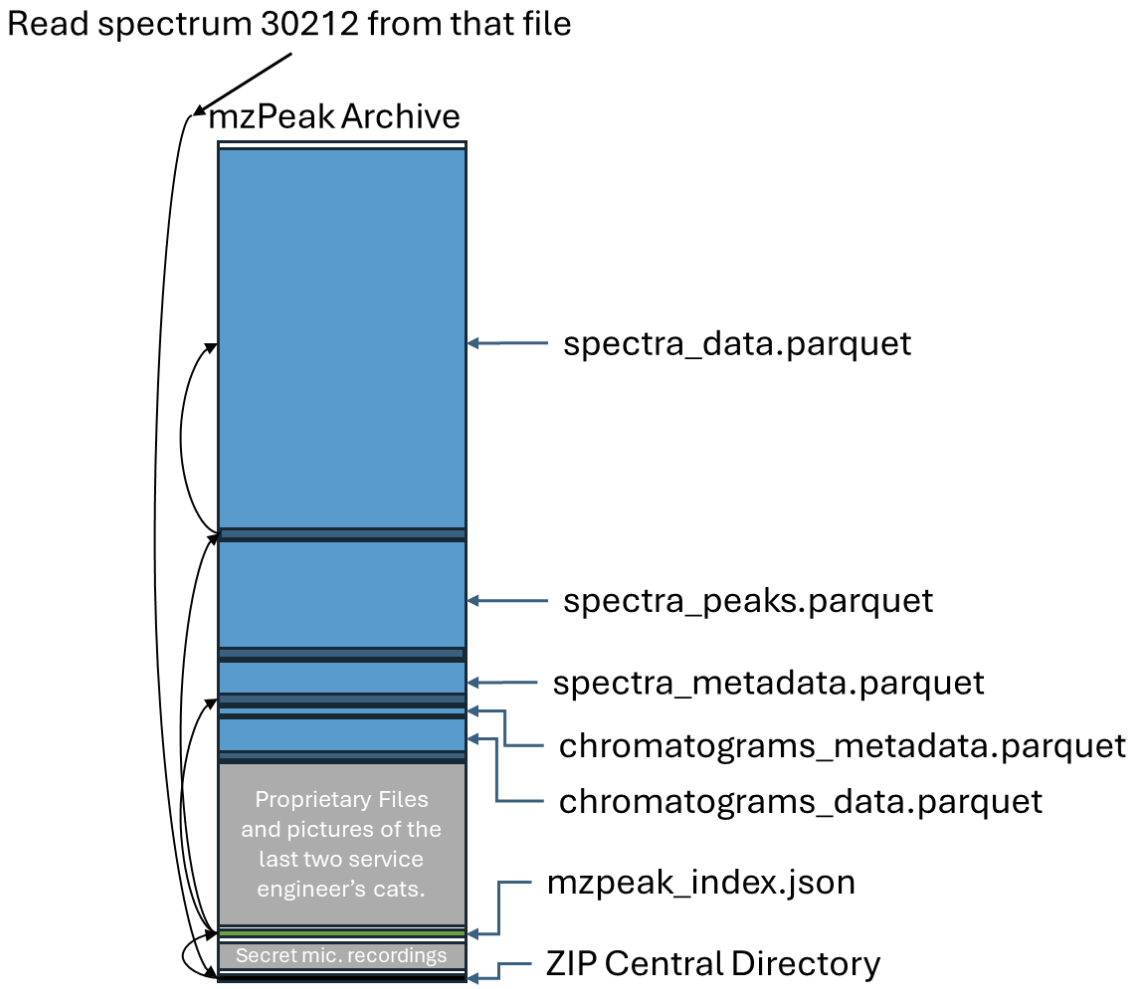
- Web apps can directly access cloud storage with immediate access to the data
- No software installation needed to quickly check (or analyze) the data
- Quick access - only the data to be explored will be loaded directly
- Several mzPeak examples and demo apps are available online
- Runs entirely in the browser - for local and remote data
- No data will be transmitted to the cloud for local data (ensures data privacy)



Online example data: https://data.mzpeak.org/v09/index.html

mzPeak Is Fast

- Parquet provides efficient blocked random access to the data
- Opening the file requires a read of the central directory of the ZIP archive (if packed), the index JSON, and then only the footers of the Parquet files for the dimensions the user is interested in, loading fewer than 30 MB from a 3 GB file
- Reads can be planned using hierarchical indices and read the minimum content required, usually a single data page per data array
- Spectrum metadata loads quickly in a single block that comes pre-packed into native types for efficient storage



mzPeak Integrates Seamlessly with Archives

- **Integration into one file** keeps data together
- Embedding of **experimental design** is possible through direct integration with SDRF-Proteomics
- Archives can **extract more metadata** from these files and thus simplify submission process for researchers
- Partial encryption of the data enables **controlled access** without further modifications



https://www.classicsforkids.com/what-is-an-archive/

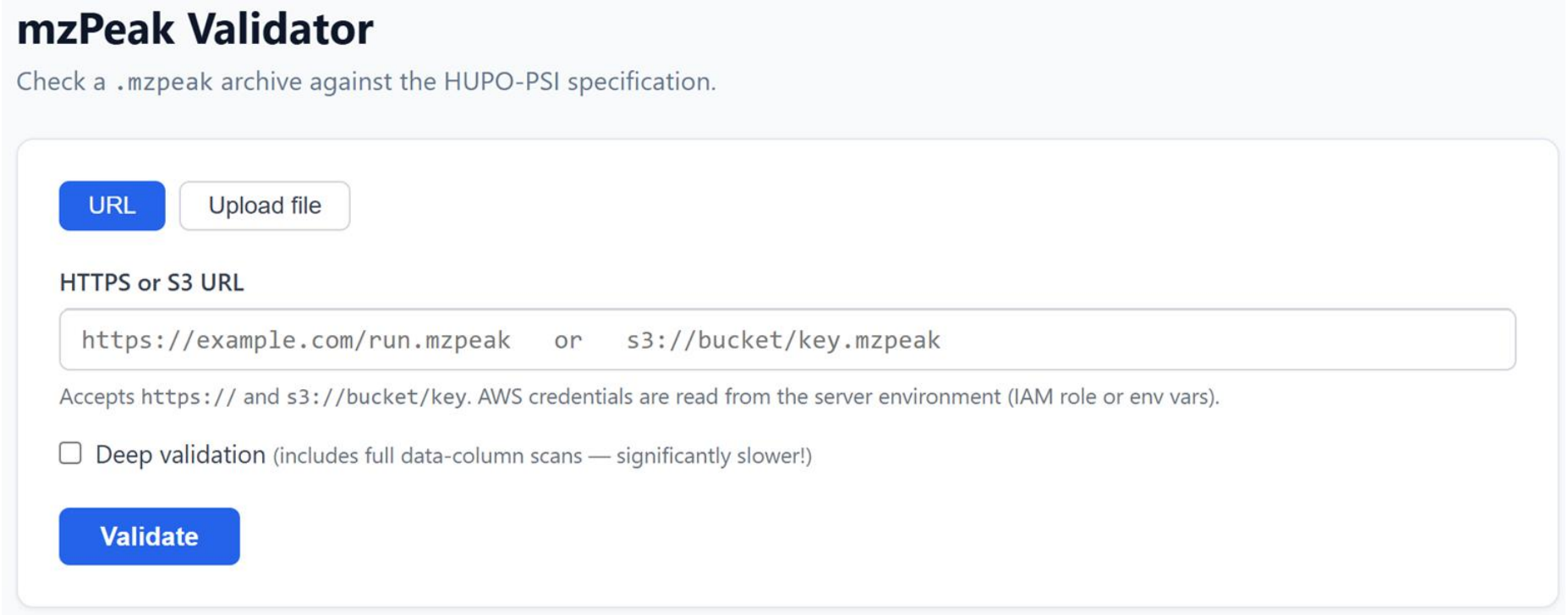
mzPeak Is Secure

- **Encryption of individual columns** enables protection of sensitive data (e.g., clinical proteomics) on a column-by-column basis
- **Encryption at rest** and centralized key management directly supported
- TLS-secured cloud transfer (S3/HTTPS) adds **encryption in transit**
- Parquet encryption supports **post-quantum safe encryption** (AES-CGM, still requires suitable key exchange mechanism)



mzPeak Can Be Easily Validated

mzPeak



Try it yourself → mzpeak.org/validator

Available Software Tools

mzPeak

mzPeakTS

TypeScript library for reading mzPeak files, with a simple demo viewer.

github.com/HUPO-PSI/mzpeakts
hupo-psi.github.io/mzpeakts/app.html

mzPeakConverter

Prototypic command line converter between vendor formats and mzPeak.

github.com/okohlbacher/mzPeakConverter

mzPeakViewer

Web-based tool for browsing and inspecting mzPeak files.

okohlbacher.github.io/mzPeakViewer

mzPeakValidator

Simple Python-based (library/command line) validator for mzPeak files.

github.com/okohlbacher/mzPeakValidator

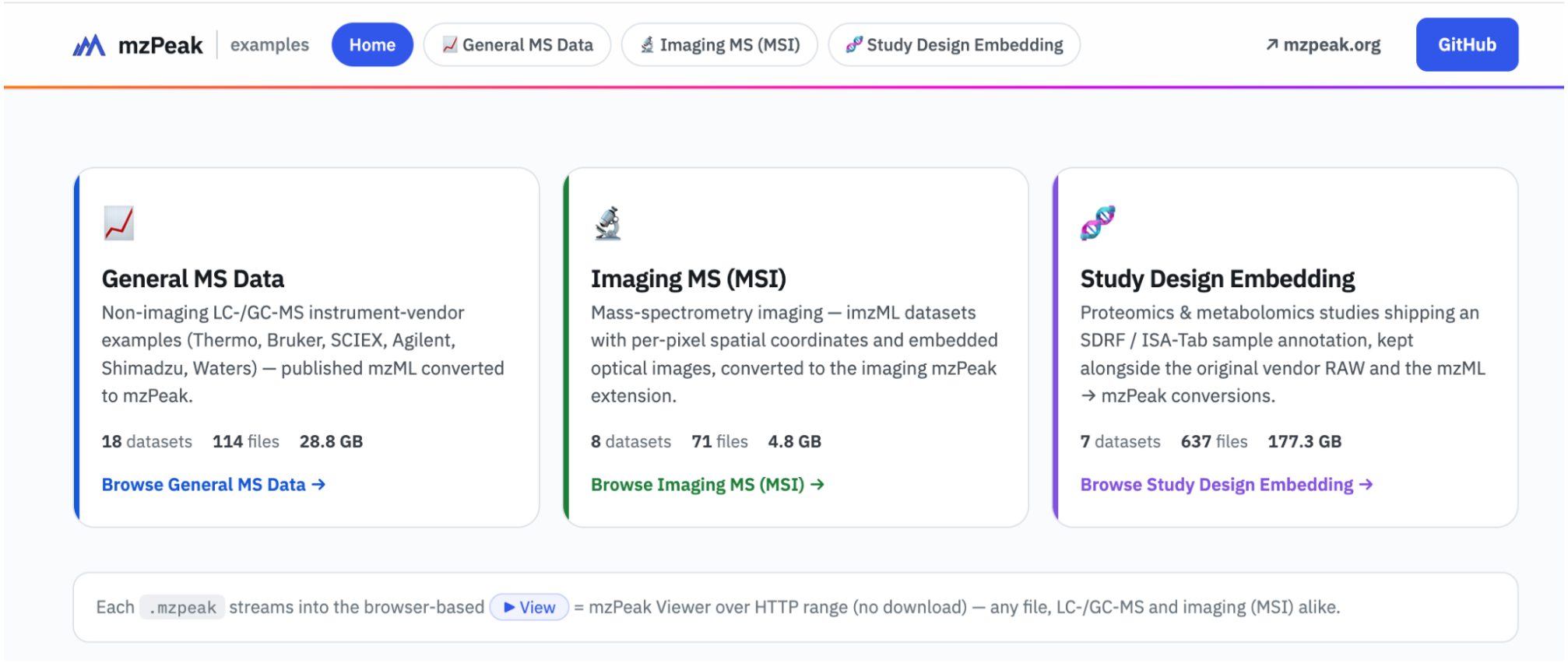
Reference Implementations of mzPeak

Currently under development for a large number of languages

Rust	mzData, Read/Write	github.com/HUPO-PSI/mzPeak
R	Read	github.com/HUPO-PSI/mzPeak/tree/main/R
C#	Read/Write	github.com/HUPO-PSI/mzPeak.NET
Python	Read	github.com/HUPO-PSI/mzPeak/tree/main/python
TypeScript	Read	github.com/HUPO-PSI/mzpeakts
Java	mzPeakJ, Read/Write	github.com/okohlbacher/mzPeakJ
C++	OpenMS, Read	github.com/OpenMS/mzpeak

mzPeak Example Files

We have assembled many publicly accessible datasets from a variety of instruments and provide the data in the respective vendor format, mzML, and mzPeak for comparison.



mzPeak

Additional Resources

mzPeak Website: https://mzpeak.org

mzPeak Whitepaper: https://pubs.acs.org/doi/10.1021/acs.jproteome.5c00435

mzPeak draft specification: https://mzpeak.org/spec

Example data: https://data.mzpeak.org/v09/index.html

Interact with the mzPeak community

Contact us: **info@mzpeak.org**